

Learn Home Automation with Arduino

Lesson 2: Arduino Basics & IDE Setup

Arduino Basics & IDE Setup – Study Notes

Key Concepts

1. **Arduino IDE installation & configuration** – downloading, installing, selecting board and port.
2. **Structure of a sketch** – `setup()`, `loop()`, includes, and basic syntax.
3. **Digital I/O pins** – pin numbering, INPUT vs. OUTPUT, using `pinMode()`, `digitalWrite()`, `digitalRead()`.
4. **Serial communication** – opening the Serial Monitor, `Serial.begin()`, `Serial.print()/println()`.
5. **The “Blink” sketch** – the classic first program that toggles an LED.

Summary

The Arduino ecosystem starts with the Arduino Integrated Development Environment (IDE), a free cross platform editor that compiles C/C++ code (called a *sketch*) and uploads it to an Arduino board via USB. After installing the IDE, you must select the correct board type (e.g., **Arduino Uno**) and the COM port that the board is using.

A sketch is composed of two mandatory functions: `setup()`—run once at power up—to configure pins, start serial communication, etc., and `loop()`—executed repeatedly—for the main program logic. The first practical sketch is **Blink**, which sets the built in LED (pin 13 on most boards) as an output and toggles it on and off with a delay. This simple program introduces digital pin control, timing with `delay()`, and the workflow of writing, verifying, and uploading code.

The Serial Monitor, accessed from the IDE, allows you to view text sent from the board with `Serial.print()/println()`. It is invaluable for debugging and for sending data back to the computer. Understanding how to open the monitor, set the baud rate, and interpret the output completes the basic development loop for beginners.

Important Points

- **IDE download:** <https://www.arduino.cc/en/software> – choose Windows, macOS, or Linux.
- **First time setup:**

1. Install the IDE.
2. Connect the Arduino board via USB.
3. In **Tools !> Board** select your board (e.g., “Arduino Uno”).
4. In **Tools !> Port** choose the detected COM/tty port.

- **Sketch skeleton:**

```
```cpp
```

```
void setup() {
```

```

// runs once
}
void loop() {
 // runs forever
}

```

- **Pin modes:**
  - `pinMode(pin, OUTPUT);` – drive a pin high/low.
  - `pinMode(pin, INPUT);` – read a pin's voltage.
  - `pinMode(pin, INPUT_PULLUP);` – enable internal pull up resistor.
- **Digital I/O:**
  - `digitalWrite(pin, HIGH/LOW);` – set output level.
  - `int state = digitalRead(pin);` – read input level.
- **Built in LED:**
  - Most boards expose an LED on **pin 13** (or the **L** symbol).
- **Serial basics:**
  - `Serial.begin(9600);` – start communication at 9600 /baud.
  - `Serial.print("msg");` – send without newline.
  - `Serial.println("msg");` – send with newline.
- **Common errors:**
  - Wrong board or port selected !' "avrdude: stk500\_recv(): programmer is not responding".
  - Missing `;` or mismatched braces !' compile error.
  - Trying to use a pin that is not digital (e.g., analog only pin on some boards).

---

## Examples

### 1. Blink Sketch (Built in LED)

```

```cpp
// Blink – makes the onboard LED blink every second
void setup() {
  pinMode(13, OUTPUT);    // set pin 13 as an output
  Serial.begin(9600);     // optional: start serial for debugging
}
void loop() {
  digitalWrite(13, HIGH); // turn LED on
  Serial.println("LED ON");
  delay(1000);            // wait 1 second
  digitalWrite(13, LOW);  // turn LED off
  Serial.println("LED OFF");
  delay(1000);            // wait 1 second
}
---
```

Explanation: `pinMode` configures pin /13. `digitalWrite` drives the LED, while `delay(1000)` pauses the program for 1000 /ms. Serial messages appear in the Serial Monitor, confirming the timing.

2. Reading a Push Button and Echoing State

```
```cpp
const int buttonPin = 2; // connect a momentary push button to pin 2
const int ledPin = 13; // use the built in LED

void setup() {
 pinMode(buttonPin, INPUT_PULLUP); // enable internal pull up
 pinMode(ledPin, OUTPUT);
 Serial.begin(9600);
}

void loop() {
 int buttonState = digitalRead(buttonPin); // LOW when pressed
 if (buttonState == LOW) {
 digitalWrite(ledPin, HIGH);
 Serial.println("Button pressed – LED ON");
 } else {
 digitalWrite(ledPin, LOW);
 Serial.println("Button released – LED OFF");
 }
 delay(200); // debounce delay
}
```
```

Explanation: The button is wired to ground; the internal pull up keeps the pin HIGH when not pressed. Pressing the button pulls the pin LOW, which the sketch detects and mirrors on the LED while printing the state to the Serial Monitor.

Quick Review

1. **What two functions must every Arduino sketch contain, and when is each executed?**
2. **Which menu items do you use to select the correct board and COM port before uploading?**
3. **How do you configure a digital pin as an output and then set it HIGH?**
4. **What command starts serial communication at 9600 /baud, and where do you view the output?**
5. **In the Blink example, what would happen if you changed `delay(1000)` to `delay(500)`?**

Further Reading

- **Arduino digital vs. analog pins** – understanding PWM, analogRead, and pin capabilities.
- **Using external libraries** – installing and including libraries (e.g., `Servo`, `LiquidCrystal`).
- **Serial communication protocols** – deeper look at baud rates, Serial.parseInt(), and sending data from the PC to the board.
- **Basic electronics for Arduino** – resistors, LEDs, pull up/pull down, and breadboarding.
- **Debugging techniques** – using the Serial Monitor, LED indicators, and the Arduino “Print” class

for troubleshooting.

